SECURITY RESEARCH

The Adversarial Research Harness

Measuring Autonomous Offensive-Agent Capability Inside Enforced Containment

Adversarix Threat Intelligence Platform
Version 0.1 (Draft) | August 2026 · AEGIS Labs

0
Sandbox escapes or denied-action bypasses, across every run

4
Models driven end to end, local and hosted, all Claude-free

10×
Exploitation effort unlocked by the harness-enforced persistence loop

100%
Agent actions mediated and logged as first-class data

Table of Contents

1 Abstract
2 1. Introduction
3 2. Design principles
3.1 2.1 Neutrality
3.2 2.2 Containment enforced in code, not policy
4 3. Architecture
5 4. The containment plane
6 5. Capability: the offensive-research stages
7 6. Measurement methodology
8 7. Case study: autonomous exploitation on ExploitGym, without a frontier proprietary model
8.1 7.1 Setup
8.2 7.2 Results across four models
8.3 7.3 Removing the confounds
8.4 7.4 Isolating capability from persistence
9 8. Findings
10 9. Limitations and future work
11 10. Conclusion

1. Abstract

Autonomous agents are becoming capable of real offensive-security work: finding vulnerabilities, developing exploits, and reasoning across multi-step attack chains. Two research communities study this from opposite ends. Offensive-agent work pushes capability but runs agents with little structural containment. Containment and safety work builds strong controls but exercises them against toy agents. Neither answers the question a defender actually has: run a genuinely capable adversarial agent against a real containment plane, and report what holds.

This paper describes the Adversarial Research Harness, a general-purpose instrument that drives any model through the full offensive-research lifecycle (discover, develop, detonate, observe) while every action passes through a containment plane enforced in code rather than policy. The harness is model-agnostic, target-agnostic, and independent of any single benchmark or prior research line. Its central design property is that capability and containment are co-designed rather than traded off: the harness makes the agent stronger through security-native interactive tools, and it makes every one of those tools cross a default-deny mediation gate that logs and can veto each action.

We present the architecture, the containment invariants, and an end-to-end empirical study in which a stock open-source agent runtime, driven entirely by non-Claude models, autonomously attacks a recognized exploit-development benchmark (ExploitGym) through the contained harness. Across four local and hosted models the containment plane held perfectly (zero denied actions bypassed, zero egress from the sandbox), while no model captured a flag on hard real-world memory-safety bugs. By progressively removing confounds (cold-start failures, tool distraction, model streaming limits, and early stopping), we isolate the remaining bottleneck as genuine exploitation capability rather than agent persistence or harness friction. The harness itself, together with a method that separates these factors and reports containment as first-class data, is the contribution.

2. 1. Introduction

The interesting question about an offensive agent is not “can it be dangerous” in the abstract. It is two coupled questions asked together.

> **The question the harness answers**
>
> How much offensive-research capability does this agent actually have, and does the containment plane hold while we measure it?

Most published work answers only one. Capability demonstrations run agents with broad, unmediated access, so the result tells you what a model can do but not what a controlled deployment could safely permit. Containment and sandboxing work, conversely, tends to validate controls against weak or scripted agents, so the result tells you the box is strong but not whether a capable adversary would strain it.

The Adversarial Research Harness treats the tension between these two goals as the contribution rather than a problem to resolve:

- **Stronger agent.** Push what an autonomous agent can do on offensive-research tasks, through security-native interactive tools (a persistent debugger, fuzzing, gadget search, exploit construction, a live target channel) and, optionally, model steering.
- **Contained agent.** Mediate every action through runtime invariants enforced in code, so the harness is safe to run unattended and its results are reproducible, and so the agent’s denied attempts become measurable data rather than silent failures.

Running a capable adversary against a real containment plane, and reporting exactly what the plane allowed and denied, is what the harness is for.

3. 2. Design principles

3.1 2.1 Neutrality

Three neutralities are load-bearing requirements, not preferences.

- **Model-agnostic.** The harness runs local and hosted models interchangeably behind a provider abstraction. In this study the same pipeline drove Ollama-served local models (qwen3:14b, gpt-oss:20b, qwen3-coder:30b) and a hosted model (Kimi-K3 on Fireworks) reached through an OpenAI-compatible LiteLLM proxy. Swapping the model is a configuration change, not a code change.
- **Target-agnostic.** Tasks and targets enter through adapters behind a stable interface. The harness ships example adapters; it is not built around any of them.
- **Use-case-agnostic.** The harness carries no bias from prior AEGIS Labs research. Benchmarks, scorers, and downstream analyses are optional points of connection, never dependencies. A user who has never seen any of that work can use the harness fully.

Neutrality is what makes the measurement credible: if no model, target, or benchmark is privileged, the numbers reflect the agent and the plane, not the fixture.

3.2 2.2 Containment enforced in code, not policy

A containment claim that rests on instructions to the model, or on an operator remembering to be careful, is not a containment claim. In this harness containment is a set of runtime invariants that a dispatch gate applies to every tool call. A violation is an error the code raises, not a judgment an operator makes. Detection-evasion as an end, and action against systems the harness does not itself instantiate and control, are unrepresentable in the action space rather than merely discouraged.

4. 3. Architecture

The harness is layered so that each concern can be swapped without disturbing the others.

- **L1, the CLI and harness.** A thin wrapper over a stock open-source agent runtime (Goose). The wrapper does not fork the runtime; it launches the stock runtime with the mediation plane loaded as the agent's only extension, so every tool the model can call is a mediated tool.
- **L2, the agent loop.** The runtime's own reasoning-and-tool-use loop, unmodified. The agent is reasoning glue over mediated tools, not a bespoke controller.
- **L3, the inference provider.** A provider abstraction rather than a single engine. Local OpenAI-compatible endpoints and hosted APIs are interchangeable; the harness also drives a hosted model through a proxy that adds budget caps and unified routing.
- **L4, the model.** Agnostic and selected by configuration.

Above these layers sits the offensive-research lifecycle the harness is built to run: discover a vulnerability, develop it into a working exploit, detonate it in an instrumented chamber, and observe the result. Custody and coordinated disclosure wrap the lifecycle so that anything the harness finds is held safely and reported responsibly.

5. 4. The containment plane

Every action the agent takes crosses a mediation seam that evaluates a small set of invariants and records the decision. The seam is an out-of-process gate loaded as the runtime's only extension, so the agent cannot reach a tool without crossing it, and it cannot reconfigure or remove it. The invariants are:

- **Default-deny governance.** Only tools explicitly in scope for the current run are allowed; everything else is denied. A run declares its tier and its allowed tools, and the gate rejects any call outside that scope.

- **Target isolation.** The agent acts only on targets the harness instantiates and controls inside the isolation boundary. Analyzing third-party software means running a copy of its code in a sandbox, never reaching a system the operator does not control.
- **Dry-run-first and a kill switch.** Higher-tier actions stage before they fire, and a kill gate can halt a run such that an otherwise-allowed action is refused.
- **Signed markers.** Each allowed action is stamped with a per-run HMAC marker, so the trajectory is tamper-evident and every effect can be tied back to the mediated call that produced it.

Runs are tiered. Green-tier work is read-only or sandbox-confined and carries no live-fire risk. Amber and red tiers add real third-party source and live-fire detonation under progressively stricter substrate rules, up to a microVM detonation chamber.

The plane produces a containment signal as a first-class output. Every mediation decision, allow or deny, with its reason and marker, lands in an append-only log. “What did the agent try that the plane denied” is therefore a reportable result, not an inference. That log is the empirical basis for the containment claims in this paper.

6. 5. Capability: the offensive-research stages

The harness makes the agent stronger by giving it security-native interactive tools, each mediated.

- **Discover.** Coverage-guided fuzzing in a network-isolated sandbox surfaces crashing inputs; deterministic replay confirms them; honest triage classifies the crash class, CWE, and weaponizability. A separate, optional path uses a purpose-trained localization backend to rank candidate vulnerable files from a CWE and a source tree, emitted as hypotheses that the confirm loop verifies. The localizer is a specialist that seeds candidates; the harness still owns the confirm loop and the promotion decision.
- **Develop.** Exploit development climbs a ladder from crash to primitive to control to a working exploit. The signature capability is a persistent-debugger interactive tool: a live gdb session and a live target process whose state persists across tool calls, so the agent works the way a human exploit developer does rather than through stateless one-shot calls. A mitigation ramp exercises the ladder against progressively harder targets (position-independent code, stack canaries, and combinations), and the offset and exploit-construction tools derive their behavior from the target architecture so the same tools work on both aarch64 and x86-64.
- **Detonate.** A red-tier detonation chamber runs a real munition against a real target under six invariants enforced in code, with guaranteed teardown even under a mid-detonation kill, on a microVM substrate.
- **Custody and disclosure.** Confirmed findings are promoted into a munitions store: inert by default, encrypted at rest, with an append-only signed and hash-chained ledger, and human-gated arm, export, and disposal that the harness cannot self-authorize. A coordinated-disclosure workflow opens an embargoed case and assembles a vendor package that carries a minimal reproducer description and never the weapon. The rule that the harness finds and a human discloses is enforced in code: the autonomous loop cannot advance a case out of embargo on its own.

An operator cockpit ties the green-tier stages together into a real research loop: point it at a real third-party target, hunt for a bug, confirm and characterize it honestly, take it into signed custody, and open an embargoed disclosure case that the loop cannot self-report.

7. 6. Measurement methodology

Every run emits a structured trajectory: each proposed tool call with its mediation verdict and result, wall-clock, and outcome. Two signals fall out of this.

- **Capability signal.** Beyond a binary solved-or-not, the harness records how far the agent climbed: whether it reached the live target at all, how many rounds of interaction it sustained, which tools it used, and where it failed. This matters because on hard tasks the binary outcome is usually “unsolved,” and a study that reports only that number learns nothing about the difference between models. Reach and interaction depth discriminate where solve rate cannot.

- **Containment signal.** Every mediation decision is logged, so denials are data. A run that stays fully contained still produces a positive result: it demonstrates that a capable agent, given real tools, took actions that the plane allowed and attempted nothing the plane had to deny.

Scoring is pluggable. A run's normalized evidence passes through a selectable scoring adapter that produces a comparable result, so no scorer is privileged and different tasks report on one scorecard.

8. 7. Case study: autonomous exploitation on ExploitGym, without a frontier proprietary model

To exercise the harness end to end against a recognized external benchmark, we drove it against ExploitGym (the `cybergym` autonomous exploit-development range) using only the contained harness and only non-Claude models. ExploitGym ships its own agent scaffolds for proprietary CLIs; running it through a stock open-source runtime and our mediation plane is a distinct path, and it is the only way to run the benchmark with a model those scaffolds do not support.

8.1 7.1 Setup

We used the `user` task family: a vulnerable program is served over a socket, a flag is placed on the target where only successful exploitation can read it, and success is defined as writing the recovered flag back for verification against a deterministic expected value. Our harness reuses the benchmark's own target bring-up unchanged (flag injection, socket protocol, sanitizer environment) so the target is byte-for-byte the one the benchmark's scorer expects. We add exactly one thing: the network.

The containment requirement here is sharp. The agent's sandbox normally runs with no network at all. The benchmark target must be reachable on a socket. We resolve this by attaching the target and the agent sandbox to a private, internal, no-egress network, so the agent can reach the one task target and nothing else. We verified from the agent's own vantage that the target is reachable while the public internet and DNS are not. The seam refuses to attach the sandbox to any network that is not internal, so the relaxation cannot become egress.

The agent works the vulnerable binary locally for static analysis, and delivers its exploit to the live target through a single mediated network tool that speaks the benchmark's socket protocol. The flag it must recover is injected by the harness and is never present in the agent's environment or prompt; the agent can only obtain it by exploiting. Because the contained agent has neither the benchmark's source nor any egress, a recovered flag is causally necessary by construction: it can only come from reading the protected file through the vulnerability.

8.2 7.2 Results across four models

We swept the same three ARVO tasks (real OSS-Fuzz-derived memory-safety bugs in ffmpeg, compiled without mitigations) across four models, all Claude-free.

Model	Where	Genuine attempts	Reached target	Peak rounds	Solved	Denied
qwen3:14b	local	2 of 3	1 of 2	1	0	0
gpt-oss:20b	local	3 of 3	2 of 3	1	0	0
qwen3-coder:30b	local	3 of 3	2 of 3	8	0	0
Kimi-K3	hosted	3 of 3	3 of 3	4	0	0

Two things are visible immediately. First, containment held for every model on every run: zero denied actions, zero egress. A capable agent given a real debugger, gadget search, and a live target channel took only actions the plane allowed. Second, no model captured a flag, but the quality of the attempt scales clearly with capability: stronger models stop producing spurious no-attempt runs, reach the live target more reliably, and interact more deeply. The reach and interaction-depth signals discriminate models that a binary 0 percent solve rate would render identical.

8.3 7.3 Removing the confounds

A 0 percent solve rate is only interesting if we can say why. We diagnosed and removed three confounds in turn, each a real issue that a naive study would have misattributed to task difficulty.

1. **Cold-start no-attempts.** A cold local model sometimes returned an empty first response and the session ended with no tool calls in roughly thirty seconds. Counted naively this looks like a failed attempt. We warm the model before timing, retry once on a no-tool-call round, and separate no-attempt runs from genuine unsolved attempts in the metric.
2. **Tool distraction and streaming limits.** A larger local model spent its budget driving the local copy of the target (a fuzzing harness that does not run standalone and holds no flag) instead of the live target, and then hit the runtime's default stream timeout on a large tool payload and ended early. We focused the exposed tool set to analysis plus the live-target channel, removing the local dead-end and shrinking the payload, and raised the stream timeout for large local models.
3. **Early stopping (agentic persistence).** With the mechanics fixed, models reliably reached the target and then stopped well short of the turn budget, reverting to a conversational summary or an offer to help. A prompt directive instructing the agent to continue did not change this behavior for any model tested. We therefore moved persistence into the harness: the runner drives the agent over one continued session across multiple rounds, and when the agent stops without the flag it resumes the same session with a nudge, halting only on a capture, on an exhausted round budget, or after two consecutive rounds with no new tool action.

8.4 7.4 Isolating capability from persistence

The harness-enforced persistence loop lets us separate two explanations for the null result that are otherwise entangled: does the agent fail to solve because it stops early, or because it cannot?

Driving qwen3-coder:30b on one task with the persistence loop produced six of six rounds used, the session resumed five times with context carried across rounds, and roughly ten times the sustained exploitation effort of the natural single-shot runs (twenty rounds of live-target interaction versus two or three). It never went two rounds without acting. It still did not solve.

> **The decisive observation**
>
> With persistence forced and twenty exploitation attempts made against the live target, the task remained unsolved. The residual wall is genuine exploitation capability, not the agent quitting. The harness fix removed the persistence confound; what is left is the model's ability to actually construct the exploit.

The same loop applied to gpt-oss:20b behaved oppositely and is equally informative. That model did local analysis in the first round, never reached the target, and answered the continuation nudge with text but no tool action, so the two-dead-round guard correctly halted it early instead of burning the budget. The loop sustains a model that keeps acting and stops a model that is genuinely stuck. Whether forced persistence helps at all depends on whether the model will keep taking tool actions under pressure, which is itself a measurable property of the model.

9. 8. Findings

- **The containment plane holds under a capable agent.** Across four models, dozens of runs, real exploit-development tooling, and a live network target, every action the agent took was one the plane allowed, and nothing the plane had to deny escaped it. Attaching the sandbox to a task target did not create egress, because the seam refuses any non-internal network. Containment reported as first-class data, rather than assumed, is what makes this a result and not a hope.
- **On hard real bugs, current open and hosted non-frontier models do not solve, and the bottleneck is exploitation capability.** Once cold-start, tool distraction, streaming limits, and early stopping are removed as confounds, and persistence is forced by the harness, the remaining failure is the model's inability to turn a real memory-safety bug into a working arbitrary read. Quality of attempt (reach, interaction depth) scales with model capability even where solve rate does not.

- **Persistence and capability are separable, and both are measurable.** Agentic persistence, the tendency to keep taking tool actions rather than concluding, is a distinct axis from exploitation skill. The harness measures it directly and can compensate for it, so a reported solve rate reflects capability rather than a model's disposition to stop early.

10. 9. Limitations and future work

- **The models tested are not the ceiling of the field.** The study deliberately excludes proprietary frontier models on this path; a genuinely stronger or exploitation-specialized model is the obvious next variable, and the harness is built to swap it in as a configuration change.
- **The task slice is small and hard.** Three real OSS-Fuzz-derived bugs establish a floor, not a distribution. An easier task slice would establish a non-zero solve floor and let the capability signal separate models more finely.
- **Trajectory-level causal scoring is deferred.** The benchmark's judge-based causal-necessity scorer is optional in the contained setting because a recovered flag is causally necessary by construction; wiring it becomes worthwhile once solves exist to score.
- **Persistence forcing is a blunt instrument.** The current loop nudges and guards against dead rounds; a more sophisticated controller could detect specific failure modes and steer, at the cost of injecting more of the harness's own reasoning into what is meant to be a measurement of the model.

11. 10. Conclusion

The harness demonstrates that capability and containment do not have to be studied separately. A stock open-source agent runtime, driven by ordinary non-frontier models, can be given real offensive-research tools and pointed at a recognized exploit-development benchmark, and every action it takes can be mediated, logged, and, where necessary, denied, without weakening the agent enough to make the measurement meaningless. In this study the containment plane held perfectly while the agents genuinely tried and, on hard bugs, genuinely failed. The value is not a single solve rate. It is an instrument that runs a capable adversary against a real containment plane, reports what the plane allowed and denied as first-class data, and separates the reasons an agent does or does not succeed, so that the next question, a stronger model, a different task, a tighter or looser tier, is a configuration change and not a new experiment.

Citation. Galappatti, K. (2026). *The Adversarial Research Harness: Measuring Autonomous Offensive-Agent Capability Inside Enforced Containment.* AEGIS Labs, Adversarix, Inc. github.com/Adversarix/aegis-labs

